

Lab Exercise 9- Pagination, Filtering, and Sorting using Spring Boot Using IntelliJ IDEA Ultimate

1. Objective

To implement **pagination, filtering, and sorting** in a **Spring Boot REST API** using **in-memory data structures**, without using **H2, MySQL, JPA, or Spring Data**.

2. Software Requirements

- IntelliJ IDEA **Ultimate Edition**
 - JDK **17 or 21**
 - Spring Boot **3.x**
 - Maven (bundled with IntelliJ)
 - Browser / Bruno
-

3. Create Spring Boot Project in IntelliJ (IMPORTANT)

Step 3.1: Create New Project

1. Open **VS Code/ IntelliJ IDEA**
 2. Click **New Project**
 3. Select **Spring Boot**
 4. Click **Next**
-

Step 3.2: Project Metadata

Fill the details exactly:

Project: **Maven**

Language: **Java**

Spring Boot: **3.x**

Group: **com.example**

Artifact: **paginationfiltersortnodb**

Name: **paginationfiltersortnodb**

Packaging: **Jar**

Java Version: **17 (or 21)**

Click **Next**

Step 3.3: Select Dependencies

Select only:

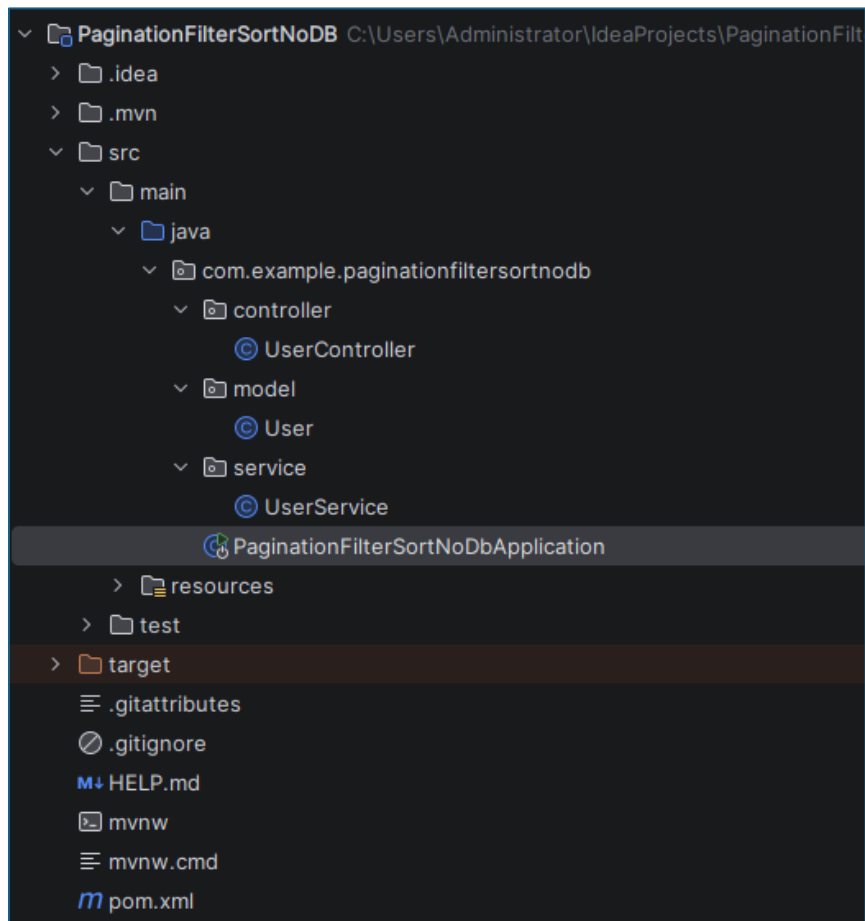
Spring Web

Click **Create**

IntelliJ will generate the project automatically.

4. Project Structure (Auto-Generated)

After creation, your project will look like:



5. Create Model Class (User)

Step 5.1: Create Package

Right-click src/main/java/com/example/paginationfiltersortnodb

→ New → Package

→ model

Step 5.2: Create User.java

src/main/java/com/example/**paginationfiltersortnodb**/model/User.java

```
package com.example.paginationfiltersortnodb.model;

public class User {

    private int id;

    private String name;

    private int age;

    public User() {}

    public User(int id, String name, int age) {

        this.id = id;

        this.name = name;

        this.age = age;

    }

    public int getId() { return id; }

    public void setId(int id) { this.id = id; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }
```

```
public int getAge() { return age; }

public void setAge(int age) { this.age = age; }

}
```

6. Create Service Layer (In-Memory Storage)

Step 6.1: Create Package

Right-click com.example.paginationfiltersortnodb

→ New → Package

→ service

Step 6.2: Create UserService.java

src/main/java/com/example/demo/service/UserService.java

```
package com.example.paginationfiltersortnodb.service;

import com.example.paginationfiltersortnodb.model.User;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class UserService {

    private final List<User> users = new ArrayList<>();
```

```
public UserService() {  
    // Sample in-memory data  
    for (int i = 1; i <= 20; i++) {  
        users.add(new User(i, "User " + i, 20 + (i % 5)));  
    }  
}  
  
public List<User> getUsers() {  
    return users;  
}  
}
```

7. Create Controller (Pagination + Filtering + Sorting)

Step 7.1: Create Package

Right-click com.example.paginationfiltersortnodb

→ New → Package

→ controller

Step 7.2: Create UserController.java

src/main/java/com/example/paginationfiltersortnodb/controller/UserController.java

```
package com.example.paginationfiltersortnodb.controller;
```

```
import com.example.demo.model.User;
```

```
import paginationfiltersortnodb.service.UserService;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import java.util.*;
```

```
@RestController
```

```
@RequestMapping("/users")
```

```
public class UserController {
```

```
    private final UserService service;
```

```
    public UserController(UserService service) {
```

```
        this.service = service;
```

```
    }
```

```
    @GetMapping
```

```
    public Map<String, Object> getUsers(
```

```
        @RequestParam(defaultValue = "") String name,
```

```
        @RequestParam(defaultValue = "0") int page,
```

```
@RequestParam(defaultValue = "5") int size,  
@RequestParam(defaultValue = "id") String sortBy,  
@RequestParam(defaultValue = "asc") String order  
) {  
  
    List<User> users = new ArrayList<>(service.getUsers());  
  
    // Filtering  
    if (!name.isEmpty()) {  
        users.removeIf(u ->  
            !u.getName().toLowerCase().contains(name.toLowerCase()));  
    }  
  
    // Sorting  
    users.sort((a, b) -> {  
        Comparable v1 = getValue(a, sortBy);  
        Comparable v2 = getValue(b, sortBy);  
        return v1.compareTo(v2);  
    });  
  
    if (order.equalsIgnoreCase("desc")) {  
        Collections.reverse(users);  
    }  
}
```



```
// Pagination

int total = users.size();

int start = page * size;

int end = Math.min(start + size, total);


List<User> paginatedList = new ArrayList<>();

if (start < total) {

    paginatedList = users.subList(start, end);

}


// Response

Map<String, Object> response = new LinkedHashMap<>();

response.put("page", page);

response.put("size", size);

response.put("total", total);

response.put("data", paginatedList);


return response;

}


private Comparable getValue(User u, String sortBy) {

    return switch (sortBy) {

        case "name" -> u.getName();

        case "age" -> u.getAge();

    };

}
```

```
        default -> u.getId();  
    };  
}  
}
```

8. Run the Application in IntelliJ

Step 8.1: Run Spring Boot App

1. Open:

PaginationFilterSortNoDBApplication.java

2. Click ► **Run**

You should see:

Tomcat started on port(s): 8080

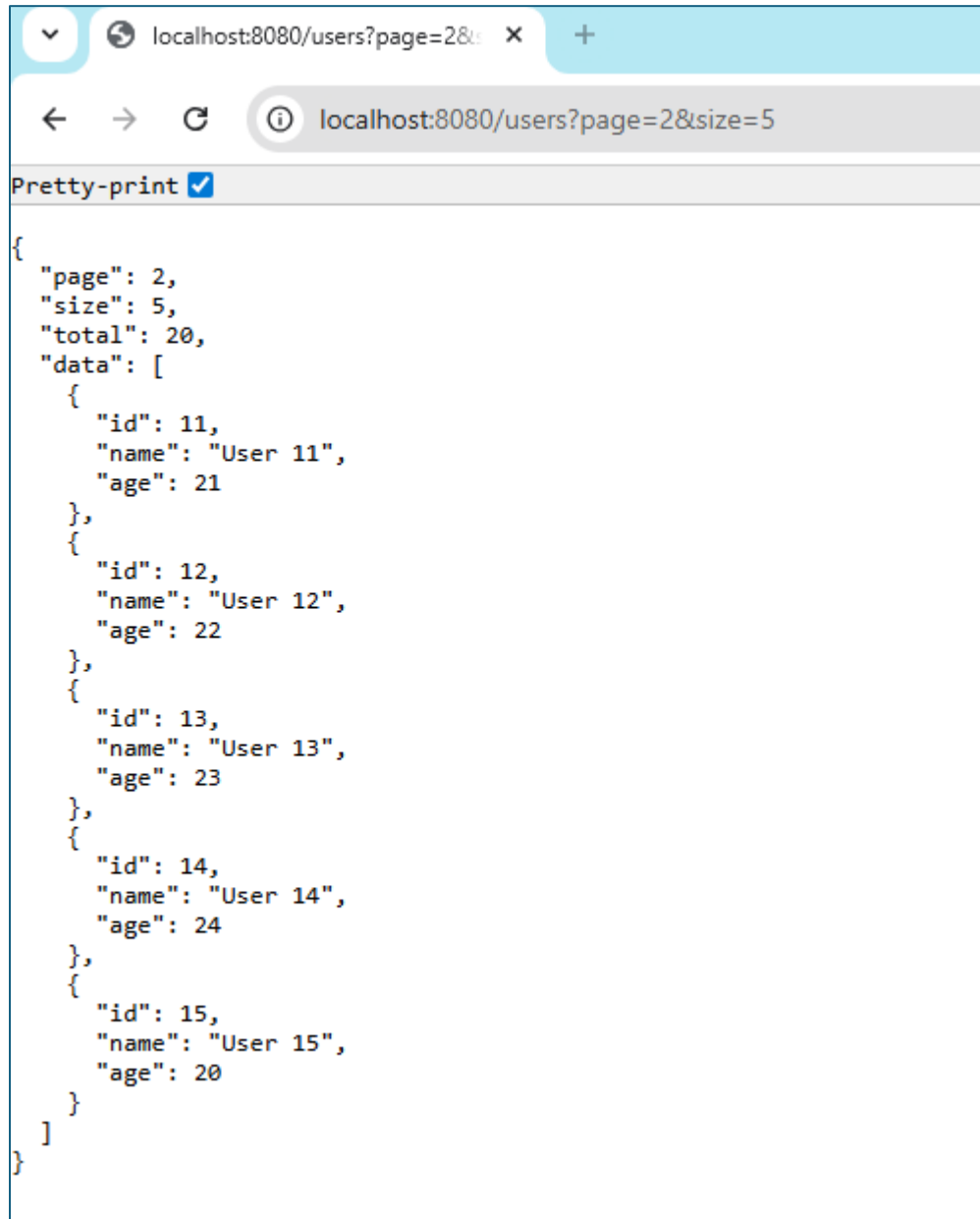
9. Test API (Browser / Postman)

Base URL

```
http://localhost:8080/users
```

A. Pagination

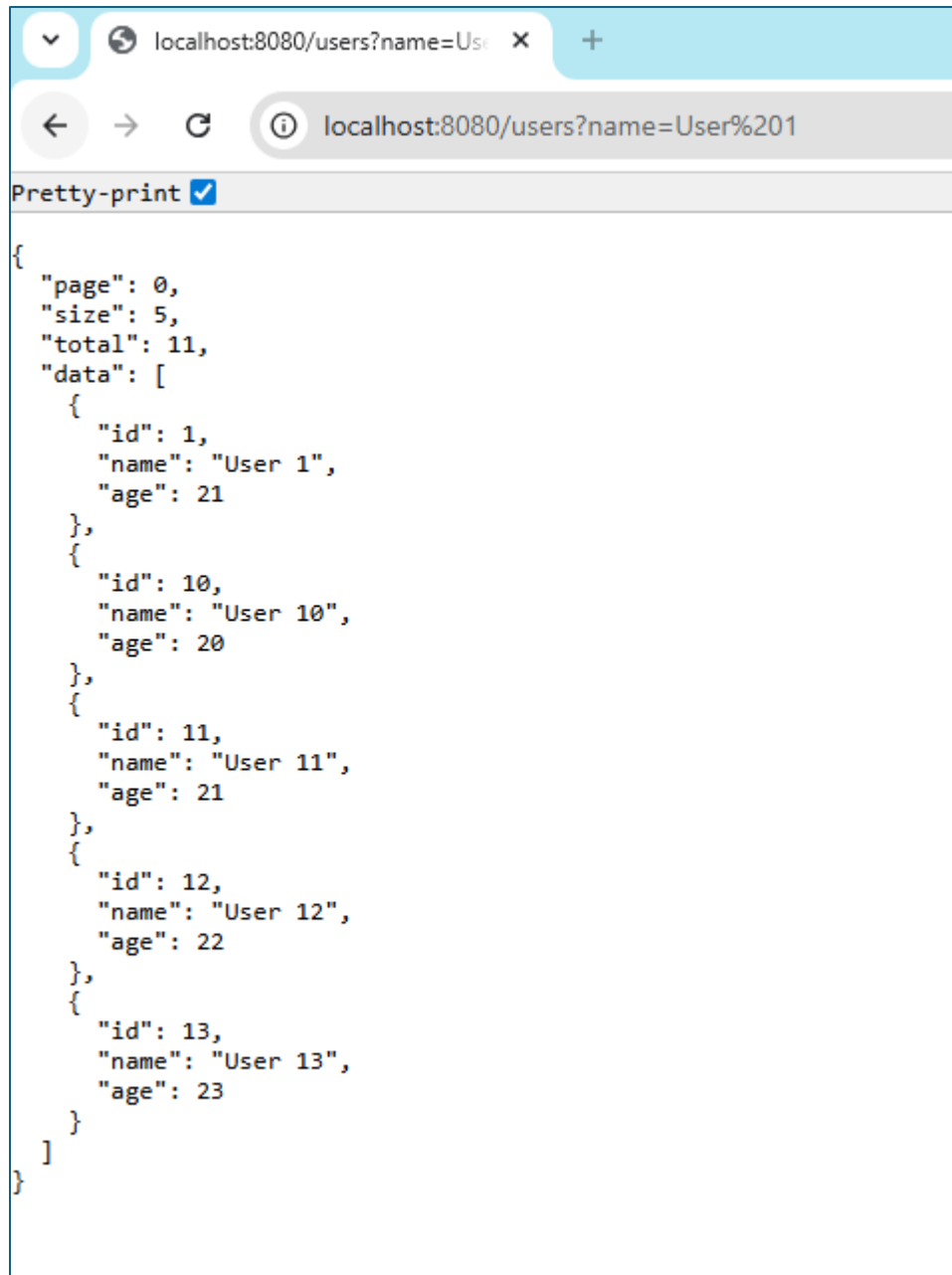
GET /users?page=2&size=5



```
{
  "page": 2,
  "size": 5,
  "total": 20,
  "data": [
    {
      "id": 11,
      "name": "User 11",
      "age": 21
    },
    {
      "id": 12,
      "name": "User 12",
      "age": 22
    },
    {
      "id": 13,
      "name": "User 13",
      "age": 23
    },
    {
      "id": 14,
      "name": "User 14",
      "age": 24
    },
    {
      "id": 15,
      "name": "User 15",
      "age": 20
    }
  ]
}
```

B. Filtering

GET /users?name=User 1



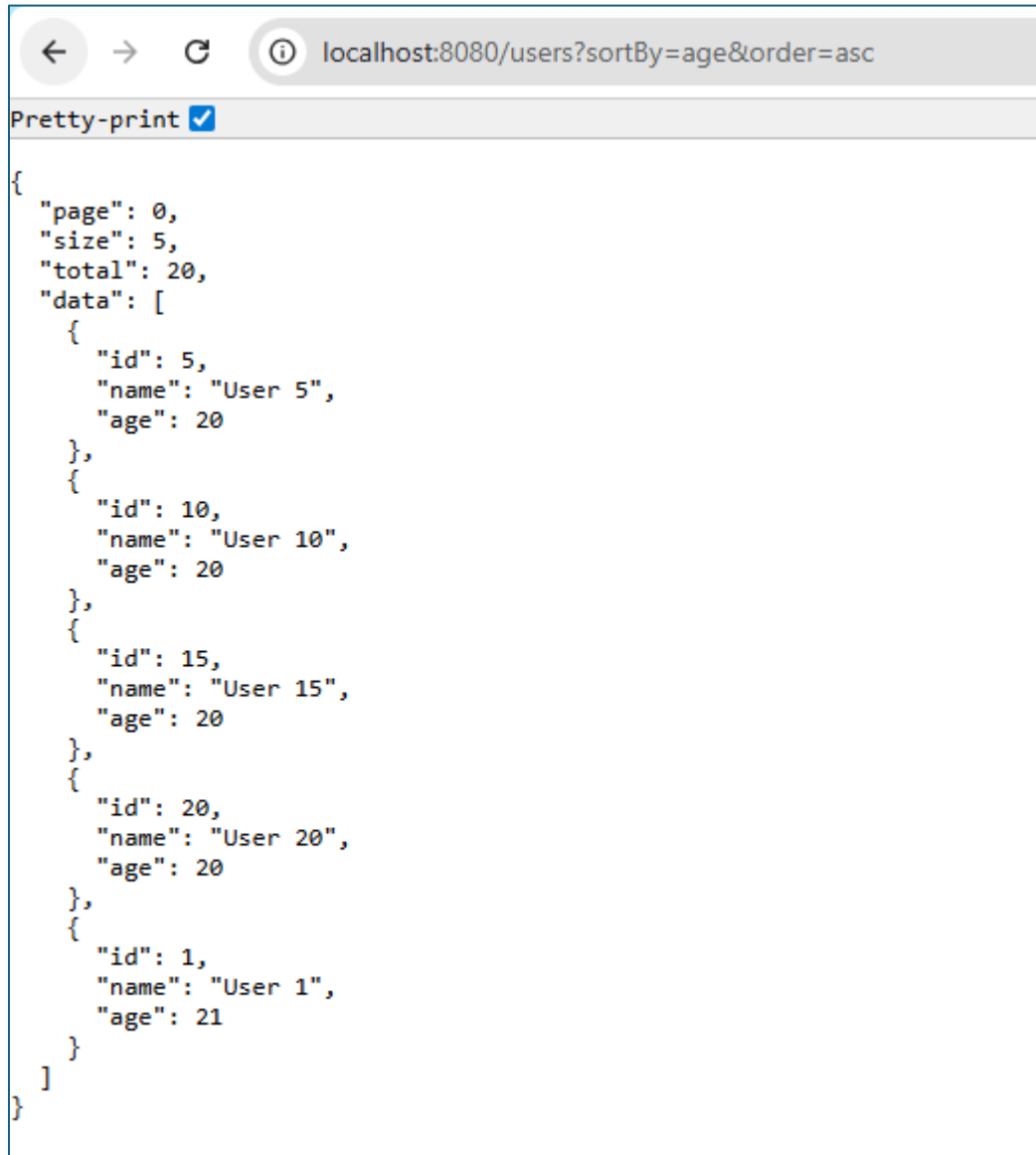
The screenshot shows a web browser window with a REST client interface. The address bar displays the URL `localhost:8080/users?name=User 1`. The response is formatted as JSON and includes a 'Pretty-print' checkbox that is checked. The JSON data is as follows:

```
{
  "page": 0,
  "size": 5,
  "total": 11,
  "data": [
    {
      "id": 1,
      "name": "User 1",
      "age": 21
    },
    {
      "id": 10,
      "name": "User 10",
      "age": 20
    },
    {
      "id": 11,
      "name": "User 11",
      "age": 21
    },
    {
      "id": 12,
      "name": "User 12",
      "age": 22
    },
    {
      "id": 13,
      "name": "User 13",
      "age": 23
    }
  ]
}
```

C. Sorting

Ascending:

```
GET /users?sortBy=age&order=asc
```



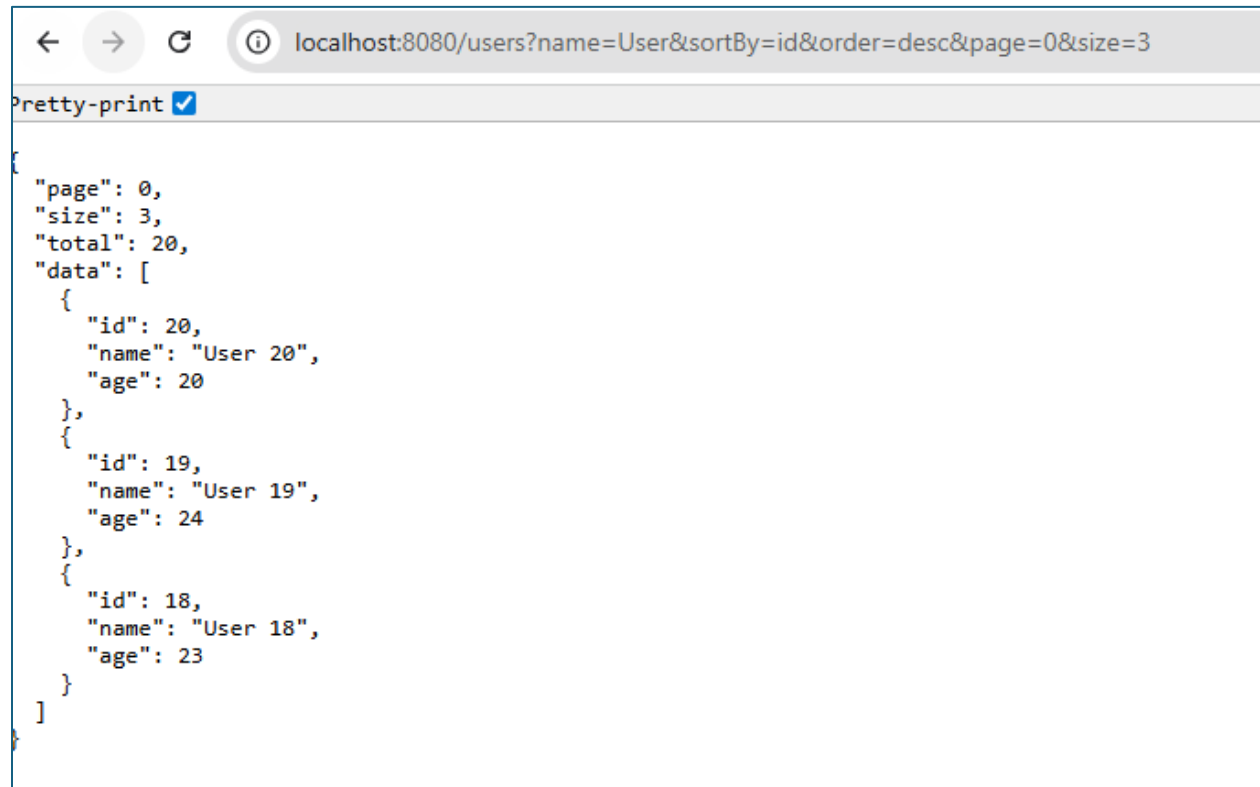
```
{
  "page": 0,
  "size": 5,
  "total": 20,
  "data": [
    {
      "id": 5,
      "name": "User 5",
      "age": 20
    },
    {
      "id": 10,
      "name": "User 10",
      "age": 20
    },
    {
      "id": 15,
      "name": "User 15",
      "age": 20
    },
    {
      "id": 20,
      "name": "User 20",
      "age": 20
    },
    {
      "id": 1,
      "name": "User 1",
      "age": 21
    }
  ]
}
```

Descending:

```
GET /users?sortBy=name&order=desc
```

D. Combined Example

GET /users?name=User&sortBy=id&order=desc&page=0&size=3



```
localhost:8080/users?name=User&sortBy=id&order=desc&page=0&size=3
Pretty-print ☒
{
  "page": 0,
  "size": 3,
  "total": 20,
  "data": [
    {
      "id": 20,
      "name": "User 20",
      "age": 20
    },
    {
      "id": 19,
      "name": "User 19",
      "age": 24
    },
    {
      "id": 18,
      "name": "User 18",
      "age": 23
    }
  ]
}
```